

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

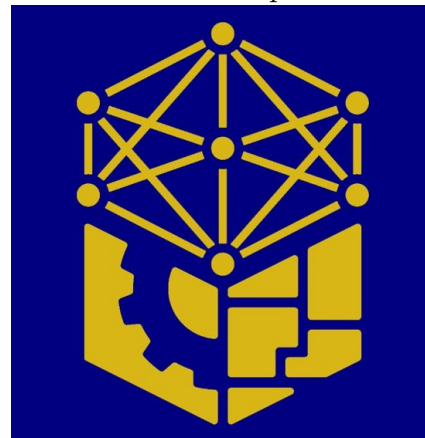
Polo Chura, Marco Rosau

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas

Índice de figuras

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 XV6

1.1.1 Nombre del proyecto o sistema operativo

El sistema operativo xv6 es una versión moderna y simplificada del Unix Sexta Edición (V6), creada con fines educativos. Fue desarrollado por el MIT en 2006 para el curso de Engineering Operating Systems (6.828) (?), con el objetivo de ofrecer una herramienta didáctica más accesible y actual que el Unix original, el cual estaba escrito en un dialecto antiguo de C (pre-ANSI). Esta reimplementación mantiene la esencia y estructura del Unix clásico, pero utiliza un código más claro y compatible con los entornos de aprendizaje contemporáneos.

1.1.2 Enlace al repositorio y/o documentación oficial

- Pagina oficial: <https://pdos.csail.mit.edu/6.828/2018/xv6.html>
- Enlace al repositorio oficial: <https://github.com/mit-pdos/xv6-public>
- Enlace al libro oficial: <https://pdos.csail.mit.edu/6.828/2018/xv6/book-rev10.pdf>

1.1.3 Objetivo del proyecto

El propósito principal de xv6 es educativo y experimental. Fue diseñado para funcionar como un kernel de estudio que permita a los estudiantes comprender los principios fundamentales de Unix dentro de un curso semestral (?). A diferencia de sistemas operativos más complejos como Linux o BSD, xv6 mantiene una estructura lo bastante simple como para ser analizada y entendida en su totalidad durante una

asignatura, pero al mismo tiempo conserva los componentes esenciales y la lógica interna de un sistema Unix real.

1.1.4 Lenguaje(s) de implementación

El núcleo de xv6 está escrito principalmente en ANSI C, con pequeñas porciones en lenguaje ensamblador (por ejemplo, para rutinas de arranque y manejo de interrupciones)(?) (pag.7). La versión original de x86 es C (para x86-32); la versión reciente soporta RISC-V también escrita en C

1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Como explica la documentación, xv6 es un kernel monolítico: toda la funcionalidad del sistema corre con privilegios de kernel, sin servidores externos (?) (pag.25) En Xv6, el kernel sigue el diseño clásico de los sistemas Unix. Funciona como un núcleo único que brinda servicios a múltiples procesos, los cuales se ejecutan en un espacio de usuario independiente. Cada proceso va alternando entre dos modos de ejecución:

- **En el modo usuario:** el proceso ejecuta su propio código y realiza operaciones computacionales básicas.
- **En el modo kernel:** es el sistema operativo quien toma el control para ejecutar las llamadas al sistema solicitadas.

Este cambio ocurre cuando un proceso necesita un servicio del sistema operativo: el hardware cambia automáticamente a un modo de mayor privilegio (modo supervisor) y transfiere la ejecución al kernel. Una vez que este completa la tarea, devuelve el control al proceso en el espacio de usuario.(?) (pag.9)

1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)

Xv6 implementa los componentes esenciales de un sistema operativo Unix clásico:

Tabla 1.1: Componentes principales del sistema operativo **XV6**

Componente / Sub-sistema	Archivo(s) principal(es)	Funcionamiento técnico (idea clave)
Gestión de procesos y planificador	<code>proc.c</code> , <code>proc.h</code>	Define la estructura <code>proc</code> y el planificador round-robin por CPU; maneja el ciclo de vida de procesos (<code>fork</code> , <code>exec</code> , <code>exit</code> , <code>wait</code>).
Gestión de memoria virtual	<code>vm.c</code> , <code>kalloc.c</code>	Implementa tablas de páginas por proceso (Sv39) y asignador físico mediante <code>kalloc/kfree</code> .
Sistema de archivos	<code>fs.c</code> , <code>file.c</code> , <code>log.c</code>	FS tipo Unix con <code>inode</code> , directorios, caché de bloques y journaling mediante write-ahead log.
Interfaz de llamadas (syscalls)	<code>syscall.c</code> , <code>sysproc.c</code> , <code>usys.S</code>	Tabla de <code>syscalls</code> y despacho de traps del modo usuario al kernel (<code>fork</code> , <code>read</code> , <code>write</code> , etc.).
subsistemas de E/S y drivers	<code>console.c</code> , <code>uart.c</code> , <code>virtio_disk.c</code>	Drivers básicos UART y VirtIO integrados con la caché de bloques.
Comunicación entre procesos (IPC)	<code>pipe.c</code> , <code>sys_pipe.c</code>	Implementa pipes unidireccionales en memoria; permite transmisión de datos entre procesos relacionados mediante <code>read/write</code> sincronizados.

Gestión de procesos y planificación

La creación de procesos usa `fork()` para copiar el proceso padre y `exec()` para cargar nuevos programas. La planificación emplea un algoritmo **round-robin** básico, donde cada CPU asigna turnos de tiempo fijo a los procesos en una lista cíclica, sin prioridades. Todos los procesos comparten el mismo entorno básico, sin distinción entre usuarios (?) (pág. 82).

Gestión de memoria virtual

Xv6 gestiona la memoria mediante paginación, asignando a cada proceso un espacio de direcciones virtual independiente. Utiliza solo 38 bits de los 64 que soporta RISC-V, permitiendo teóricamente hasta 256 GB de memoria por proceso ? (pág. 38).

El espacio de direcciones de cada proceso sigue la convención Unix clásica: La distribución de la memoria sigue el diseño clásico de Unix:

- **Segmento de Código:** Ubicado en las direcciones más bajas (cerca de 0), con permisos de solo lectura y ejecución para proteger el código del programa.
- **Datos y Heap:** Situados a continuación del código, con permisos de lectura y escritura. El heap puede expandirse dinámicamente cuando el programa solicita más memoria mediante la llamada `sbrk`.
- **Pila del Usuario:** Ubicada en la parte superior de la memoria, crece hacia abajo y tiene una *página guardia* inferior. Cualquier acceso a esta página causa que el kernel termine el proceso, previniendo desbordamientos.

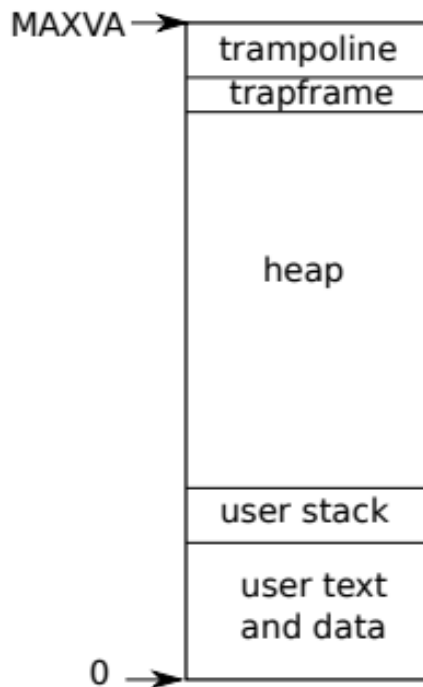


Figura 1.1: Disposición del espacio de direcciones virtuales de un proceso.

Fuente: Obtenido de (?) *xv6: a simple, Unix-like teaching operating system*.

Sistema de archivos

Xv6 implementa un sistema de archivos jerárquico que garantiza la integridad de los datos mediante journaling. Su diseño por capas organiza las funciones desde el acceso al disco hasta la gestión de archivos, haciendo que cada componente sea especializado y manejable. (?).

Organización en Disco El disco se estructura en secciones bien definidas:

- **Bloque 0:** Contiene el código de arranque
- **Bloque 1:** Almacena el “superbloque” con los metadatos del sistema de archivos

- **Bloques siguientes:** Albergan los inodos (que representan archivos)
- **Secciones posteriores:** Incluyen mapas de bits para espacios libres y los bloques de datos reales
- **Zona final:** Reservada para el journal o registro transaccional

Gestión de Archivos y Directorios Los directorios son archivos especiales que asocian nombres con inodos. Las rutas se resuelven secuencialmente (ej: `/usr/bin/ls` busca `usr`, luego `bin`).

Las operaciones básicas (`open`, `read`, `write`, `close`) usan **descriptores de archivo** y una **caché de bloques**. `read` carga datos a memoria, mientras `write` los marca para escritura transaccional. (?) (pag. 85-86)

Interfaz de llamadas (syscalls)

Xv6 implementa llamadas al sistema que siguen el modelo tradicional de Unix, permitiendo a los programas de usuario solicitar servicios del kernel mediante interrupciones de software. (?) (pág. 9)

El mecanismo funciona de la siguiente manera:

1. Un proceso en modo usuario invoca una syscall, lo que hace que el hardware genere una interrupción.
2. Esta interrupción transfiere el control al kernel, cambiando el modo de ejecución de la CPU.
3. El kernel identifica la syscall solicitada a través de un número único y ejecuta la función correspondiente.

4. Una vez completada, el kernel devuelve el control al proceso de usuario con los resultados necesarios.

E/S y drivers

Los dispositivos en xv6 se integran en el sistema bajo el principio de que “todo es un archivo”. Esto se logra mediante **archivos de dispositivo**, creados con la llamada al sistema `mknod`.

- `mknod(path, major, minor)`: Crea un archivo especial que representa un dispositivo. Los parámetros `major` (mayor) y `minor` (menor) son números que identifican de forma única al controlador (*driver*) del kernel y al dispositivo específico.

Controladores de Dispositivos (Drivers) El kernel incluye controladores mínimos pero funcionales para hardware esencial. Algunos ejemplos son:

- `uart.c` para el puerto serie.
- `virtio_disk.c` para el acceso al disco.
- `timer.c` para el temporizador del sistema.

Comunicación entre procesos (IPC)

Xv6 implementa un mecanismo básico de comunicación entre procesos (IPC) mediante **pipes**. Las pipes son canales unidireccionales en memoria que permiten a dos procesos relacionados intercambiar datos de forma sincronizada.(?)(pag. 74)

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Para la construcción, compilación y ejecución de *xv6*, se emplean las siguientes herramientas principales:

- **GCC (GNU C Compiler):** Compila el kernel y los programas de usuario escritos en C, generando binarios ELF compatibles con las arquitecturas x86-32 o RISC-V.
- **QEMU:** Emulador de hardware ampliamente utilizado para ejecutar *xv6* en entornos virtuales. Permite probar el sistema operativo sin necesidad de hardware físico, simulando arquitecturas x86 o RISC-V.
- **Cross-compiler:** En sistemas como macOS o Windows, se requiere un compilador cruzado para generar binarios ELF destinados a la arquitectura objetivo de *xv6*.

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Xv6 fue diseñado específicamente para ser un sistema operativo ligero y educativo. Su código completo consta de aproximadamente 10,000 líneas (equivalente a unas 99 páginas impresas) Su simplicidad relativa (ausencia de capas complejas como módulos dinámicos) lo hace muy accesible para estudiantes de OS. Aun así, cubre los conceptos esenciales de concurrencia, paginación, sistemas de archivos y llamadas al sistema típicos de un Unix real(?) (pag.10)

1.2 freeRTOS

1.2.1 Nombre del proyecto o sistema operativo

FreeRTOS — Kernel de tiempo real (RTOS) de código abierto orientado a microcontroladores y microprocesadores de recursos limitados. Su historia y mantenimiento han estado ligados a Richard Barry / Real Time Engineers Ltd., y desde 2017/2018 AWS mantiene y distribuye componentes y bibliotecas asociadas (Amazon FreeRTOS/AWS FreeRTOS).(?)wikipedia-FreeRTOS) sistema operativo en tiempo real basado en UNIX, diseñado para ser pequeño y eficiente, ideal para sistemas embebidos y dispositivos con recursos limitados.

1.2.2 Enlace al repositorio y/o documentación oficial

- **Pagina oficial:** <https://www.freertos.org/?>
- **Enlace al repositorio oficial:** <https://github.com/FreeRTOS/FreeRTOS?>
- **Enlace al libro oficial:** https://www.freertos.org/media/2018/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf?

1.2.3 Objetivo del proyecto

Educativo, experimental, comercial

- FreeRTOS es un sistema operativo de tiempo real (RTOS) para sistemas embebidos, concebido para ser muy liviano y portable. Está licenciado

bajo MIT (código abierto).

- Su objetivo es proveer un kernel RTOS completo (planificador, comunicación entre tareas, temporizadores, etc.) para microcontroladores y pequeños procesadores.
- Es usado tanto en entornos industriales/hobby como en educación (se emplea como ejemplo real en cursos de sistemas embebidos).

(?, pag. 19)

1.2.4 Lenguaje(s) de implementación

El núcleo de FreeRTOS está escrito principalmente en C (para facilitar portabilidad) con pequeñas partes en ensamblador según la arquitectura (rutinas de cambio de contexto)(?),

1.2.5 Arquitectura del sistema (monolítica, microkernel, etc.)

- FreeRTOS tiene una arquitectura de tipo **microkernel**, es decir, el núcleo es muy reducido y solo gestiona lo esencial (planificación de tareas, comunicación entre procesos y gestión de memoria).
- El sistema se organiza en **capas**:
 - * Una capa independiente de hardware (código común para todas las arquitecturas)

- * Una capa dependiente de hardware (abstracción para cada CPU/compilador)
- * Por encima de ellas el código de usuario (tareas e ISRs)

(?, pag. 38-40) la siguiente figura muestra las capas de software: ??.

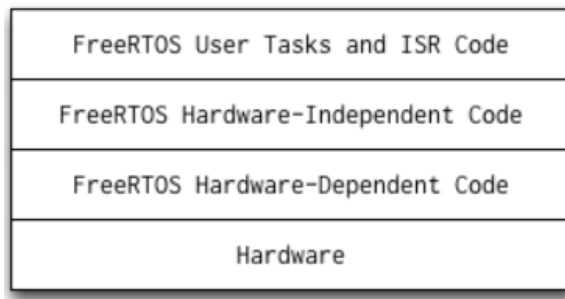


Figura 1.2: Capas de FreeRTOS (usuario/ISRs, núcleo independiente de HW, capa dependiente de HW, hardware)

Fuente: Obtenido de (?, pag. 40) *The Architecture of Open Source Applications, Volume*

II.

1.2.6 Componentes implementados (procesos, memoria, archivos, etc.)

FreeRTOS es un RTOS minimalista, por lo que implementa principalmente:

Tabla 1.2: Componentes principales de FreeRTOS

Componente / Sub-sistema	Nombre en FreeRTOS	Funcionamiento técnico (idea clave)
Gestión de tareas (threads)	tasks.c, list.c	Creación y planificación de tareas con prioridades preemptivas. Manejo de cambio de contexto en cada tick del sistema.
Comunicación / sincronización	queue.c, queue.h	Colas de mensajes FIFO, semáforos binarios y mutexes para sincronizar tareas/ISRs.
Temporización	timers.c	Temporizadores por software (delay, callbacks periódicos) con tarea de servicio propio. Tick del sistema configurable (10-1000 Hz).
Gestión de memoria	heap_1.c a heap_5.c	Múltiples esquemas de heap para asignación dinámica. Sin memoria virtual ni paginación.
Interrupciones	port*.c	Mínimo soporte en núcleo: tick del hardware y enmascaramiento de interrupciones críticas. APIs especiales para ISRs.
Eventos y temporales avanzados	event_groups.c	Grupos de flags (eventos) para sincronización múltiple entre tareas.

Gestión de tareas

Las tareas son las unidades básicas de ejecución en FreeRTOS, gestionadas por el planificador del sistema. Cada una posee su propia pila (*stack*) y estructura de control (TCB). Se crean mediante `xTaskCreate()` y se sincronizan usando los mecanismos del kernel.(?)

Comunicación / sincronización

FreeRTOS proporciona diversos mecanismos para la coordinación entre tareas e interrupciones. Entre estos se incluyen colas (`xQueueSend/xQueueReceive`), semáforos (binarios y de conteo), mutexes y grupos de eventos. Estas primitivas constituyen la base fundamental para la comunicación y sincronización dentro del sistema.(?)

Temporización

FreeRTOS incorpora temporizadores software que permiten programar la ejecución de funciones callback después de un intervalo específico o de forma periódica. Estos timers son gestionados a través de una API especializada que facilita la implementación de acciones temporizadas dentro de las aplicaciones.(?)

Gestión de memoria

FreeRTOS dispone de cinco implementaciones de memoria dinámica (`heap_1` a `heap_5`) con diferentes estrategias de asignación. Estas van desde métodos

básicos hasta esquemas avanzados que gestionan la fragmentación. La selección permite adaptarse a los requerimientos específicos de cada aplicación embebida.(?)

Interrupciones

FreeRTOS gestiona las interrupciones mediante archivos `port*.c` con soporte mínimo en el núcleo. Se encarga del tick del sistema y del enmascaramiento de interrupciones críticas. Proporciona APIs específicas para una comunicación segura entre las ISRs y las tareas.(?)

Eventos y temporales avanzados

La gestión de eventos avanzados se realiza mediante `event_groups.c`, que implementa grupos de flags para sincronización múltiple entre tareas. Este mecanismo permite que varias tareas esperen o señalen múltiples condiciones de forma eficiente, facilitando la coordinación de operaciones complejas en el sistema.

1.2.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Compiladores

FreeRTOS soporta prácticamente cualquier herramienta estándar del ecosistema embebido:

- GCC (`arm-none-eabi-gcc`, AVR-GCC)

- IAR Embedded Workbench
- Keil uVision (ARMCC)
- CodeWarrior
- Microchip XC

En general, se utiliza el *toolchain* propio de la plataforma objetivo.

Simuladores y Emuladores

- Puertos para Windows (Visual C++) y POSIX (Linux) que permiten ejecutar aplicaciones FreeRTOS en simulación.
- Demos en QEMU para varias arquitecturas (ARM, RISC-V, etc.).
- Demos preconfiguradas para placas reales (STM32, ESP32, etc.).

(?, pag. 40–41).

1.2.8 Nivel de complejidad y accesibilidad para estudiantes

FreeRTOS posee un kernel minimalista que facilita su aprendizaje inicial al ser más simple que sistemas como Linux. Sin embargo, su uso efectivo como sistema de tiempo real exige dominio de programación en C y arquitectura de microcontroladores para manejar concurrencia e interrupciones.

El proyecto ofrece documentación extensa y ejemplos prácticos como "blinky" que facilitan el aprendizaje. Es adecuado para estudiantes avanzados con bases

en sistemas embebidos, aunque dominar sus conceptos avanzados representa un desafío formativo significativo. (?).

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

Amy Brown, Greg Wilson (2014). The Architecture of Open Source Applications, Volume II. accessed: 22-septiembre-2025.

Cox, R. and Kaashoek, F. and Morris, R. (2024). xv6: a simple, Unix-like teaching operating system. Accedido: 21 septiembre 2025.

Nannan He, Han-Way Huang (2020). USE OF FreeRTOS IN TEACHING A REAL-TIME EMBEDDED SYSTEMS DESIGN COURSE. accessed: 22-septiembre-2025.

Pekopeko11 (2024). Xv6 File System and Log Design. accessed: 21-septiembre-2025.

Richard Barry (2018a). Mastering the FreeRTOS Real Time Kernel - A Hands-On Tutorial Guide. accessed: 22-septiembre-2025.

Richard Barry (2018b). *Mastering the FreeRTOS Real Time Kernel - A Hands-On Tutorial Guide*. Real Time Engineers Ltd. For FreeRTOS V10.0.0. Accedido el 20 de septiembre de 2025.

Wikipedia contributors (2024a). FreeRTOS. accessed: 22-septiembre-2025.

Wikipedia contributors (2024b). Xv6. accessed: 21-septiembre-2025.